i got this now. from drafter import *
from dataclasses import dataclass

set_website_title("Bitcoin Mining Decentralization Analyzer")
set_site_information(
    author="Jemuel Picol",
    description="Explore how bitcoin mining is distributed among mining pools and analyze mining concentration.",
    sources="Thanks to the Drafter docs and my study group.",
    planning=["CS1 Final Project 0 Pick a problem", "CS1 Final Project 1 Design"],
    links=[

"https://github.com/me50/jem-uel/blob/cs50/problems/2022/python/meal/CS1%20Final%20Project%200_%20Pick%20a%20Problem.pdf",

"https://github.com/me50/jem-uel/blob/cs50/problems/2022/python/meal/CS1%20Final%20Project%201_%20Design.docx"
        ]
)
hide_debug_information()
set_website_framed(False)

@dataclass
class State:
    top_count: int
    threshold: float

@route
def index(state: State) -> Page:
    return Page(state, [
        "Bitcoin Mining Decentralization Analyzer\n",
        "Explore how bitcoin mining is distributed among mining pools and analyze mining concentration.",
        Button("Start Analysis", "configure"),
    ])

@route

```
def configure(state:State) ->Page:
    return Page(state,[
        "Configure your Analysis",

        "Numer of largest mining pools:",
        TextBox("top_count", state.top_count),

        "Concentration threshold (%):",
        TextBox("threshold", state.threshold),

        Link("Return Home", "return_home"),
        Button("Run Analysis", "analyze"),
        Button("Reset Settings", "reset_settings"),
    ])

@route
def analyze(state:State, top_count:int, threshold:float)->Page:
    state.top_count = top_count
    state.thredhold = threshold

    return Page(state,[
        "Your Analysis Settings\n",
        "Top pools: " + str(state.top_count),
        "Threshold: " + str(state.threshold) + "%",
        Button ("Change Settings", "configure"),
    ])
@route
def return_home(state:State, top_count:int, threshold:float)->Page:
    return index(state)

@route
def reset_settings(state: State, top_count: int, threshold: float) -> Page:
    state.top_count = 3
    state.threshold = 50.0
    return configure(state)

@route
def results(state:State)-> Page:
    pass
```

```
@route
def pool_details(state:State)-> Page:
    pass

@route
def about(state:State)->Page:
    pass
```

```
start_server(State(3,50.0))
```

...



[redacted]

[redacted]

[redacted]

```
state.top_count = top_count
```

```
state.thredhold = threshold
```

[redacted]

```
@dataclass
```

```
class State:
```

```
  top_count: int
```

```
  threshold: float
```

[redacted]

[redacted]

[redacted]

```
state.threshold = threshold
```

[redacted]

[redacted]

- Top pools: `5`
- Threshold: `60`

[redacted]

[redacted]





+1

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

```
@dataclass
```

```
class MiningPool:
```

```
  name: str
```

```
  # Add the other three attributes here
```

[redacted]

[redacted]

[redacted]



[redacted]



ive created the dataclass now and the typo, whats the next step?

[redacted]

[redacted]

[redacted]

[redacted]

```
@dataclass
```

```
class State:
```

```
  top_count: int
```

```
  threshold: float
```

[redacted]

[redacted]

```
numbers: list[int]
```

[redacted]

[redacted]

```
  pools: list[MiningPool]
```

[redacted]

[redacted]

[redacted]

```
@dataclass
```

```
class MiningPool:
```

```
  name: str
```

```
  blocks_mined: int
```

```
  period: str
```

```
  share: float
```



[redacted]

```
MiningPool("Pool A", 120, "Sample Week", 0.0)
```

[redacted]

[redacted]

[redacted]

[redacted]

```
start_server(State(3, 50.0))
```

[redacted]

[redacted]

[redacted]

```
start_server(State(3, 50.0, [
```

```
  MiningPool("Pool A", 120, "Sample Week", 0.0),
```

```
]))
```

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

```
MiningPool("AntPool", 150, "Weekly", 0.0)
```

[redacted]

[redacted]



i got this right now from drafter import *
from dataclasses import dataclass

set_website_title("Bitcoin Mining Decentralization Analyzer")
set_site_information(
    author="Jemuel Picol",
    description="Explore how bitcoin mining is distributed among mining pools and analyze mining concentration.",
    sources="Thanks to the Drafter docs and my study group.",
    planning=["CS1 Final Project 0 Pick a problem", "CS1 Final Project 1 Design"],
    links=[

"https://github.com/me50/jem-uel/blob/cs50/problems/2022/python/meal/CS1%20Final%20Project%200_%20Pick%20a%20Problem.pdf",

"https://github.com/me50/jem-uel/blob/cs50/problems/2022/python/meal/CS1%20Final%20Project%201_%20Design.docx"
        ]
)
hide_debug_information()
set_website_framed(False)

@dataclass
class MiningPool:
    name: str
    blocks_mined: int
    period: str
    share: float

@dataclass
class State:
    top_count: int
    threshold: float
    pools: list[MiningPool]

@route

```
def index(state: State) -> Page:
    return Page(state, [
        "Bitcoin Mining Decentralization Analyzer\n",
        "Explore how bitcoin mining is distributed among mining pools and analyze mining
concentration.",
        Button("Start Analysis", "configure"),
    ])



@route
def configure(state:State) ->Page:
    return Page(state,[
        "Configure your Analysis",

        "Numer of largest mining pools:",
        TextBox("top_count", state.top_count),

        "Concentration threshold (%):",
        TextBox("threshold", state.threshold),

        Link("Return Home", "return_home"),
        Button("Run Analysis", "analyze"),
        Button("Reset Settings", "reset_settings"),
    ])

@route
def analyze(state:State, top_count:int, threshold:float)->Page:
    state.top_count = top_count
    state.threshold = threshold

    return Page(state,[
        "Your Analysis Settings\n",
        "Top pools: " + str(state.top_count),
        "Threshold: " + str(state.threshold) + "%",
        Button ("Change Settings", "configure"),
    ])
@route
def return_home(state:State, top_count:int, threshold:float)->Page:
    return index(state)
```

```
@route
def reset_settings(state: State, top_count: int, threshold: float) -> Page:
    state.top_count = 3
    state.threshold = 50.0
    return configure(state)

@route
def results(state:State)-> Page:
    pass

@route
def pool_details(state:State)-> Page:
    pass

@route
def about(state:State)->Page:
    pass

start_server(State(3,50.0, [
    MiningPool("Pool A", 120, "Sample Week", 0.0),
    ]))
```

what other five mining pools should i add?

...



[redacted]

```
MiningPool("Foundry USA", 100, "Sample Week", 0.0),
```

```
MiningPool("AntPool", 90, "Sample Week", 0.0),
```

```
MiningPool("F2Pool", 75, "Sample Week", 0.0),
```

```
MiningPool("ViaBTC", 65, "Sample Week", 0.0),
```

```
MiningPool("Binance Pool", 50, "Sample Week", 0.0),
```

[redacted]

[redacted]

[redacted]

```
MiningPool("Pool A", 120, "Sample Week", 0.0),
```

[redacted]

[redacted]

[redacted]

[redacted]





[redacted]

```
MiningPool("SpiderPool", 50, "Sample Week", 0.0),
```

[redacted]



[redacted]

```
def calculate_total_blocks(pools: list[MiningPool]) -> int:
```

```
  total = 0
```

```
# Write a for loop here
```

```
  return total
```

[redacted]





i did some on top of this. from drafter import *
from dataclasses import dataclass

set_website_title("Bitcoin Mining Decentralization Analyzer")
set_site_information(
    author="Jemuel Picol",
    description="Explore how bitcoin mining is distributed among mining pools and analyze mining concentration.",
    sources="Thanks to the Drafter docs and my study group.",
    planning=["CS1 Final Project 0 Pick a problem", "CS1 Final Project 1 Design"],
    links=[

"https://github.com/me50/jem-uel/blob/cs50/problems/2022/python/meal/CS1%20Final%20Project%200_%20Pick%20a%20Problem.pdf",

"https://github.com/me50/jem-uel/blob/cs50/problems/2022/python/meal/CS1%20Final%20Project%201_%20Design.docx"
        ]
)
hide_debug_information()
set_website_framed(False)

@dataclass
class MiningPool:
    name: str
    blocks_mined: int
    period: str
    share: float

@dataclass
class State:
    top_count: int
    threshold: float
    pools: list[MiningPool]

def calculate_total_blocks(pools: list[MiningPool]) -> int:
    total = 0

```
    for pool in pools:
        total = total + pool.blocks_mined
    return total

def calculate_pool_shares(pools: list[MiningPool]) -> None:
    total_blocks = calculate_total_blocks(pools)

    for pool in pools:
        if total_blocks > 0:
            pool.share = (pool.blocks_mined / total_blocks) * 100
        else:
            pool.share = 0.0

@route
def index(state: State) -> Page:
    return Page(state, [
        "Bitcoin Mining Decentralization Analyzer\n",
        "Explore how bitcoin mining is distributed among mining pools and analyze mining
concentration.",
        Button("Start Analysis", "configure"),
    ])



@route
def configure(state:State) ->Page:
    return Page(state,[
        "Configure your Analysis",

        "Numer of largest mining pools:",
        TextBox("top_count", state.top_count),

        "Concentration threshold (%):",
        TextBox("threshold", state.threshold),

        Link("Return Home", "return_home"),
        Button("Run Analysis", "analyze"),
        Button("Reset Settings", "reset_settings"),
    ])
```

```
@route
def analyze(state:State, top_count:int, threshold:float)->Page:
    state.top_count = top_count
    state.threshold = threshold

    total_blocks = calculate_total_blocks(state.pools)
    calculate_pool_shares(state.pools)

    pool_results = []

    for pool in state.pools:
        pools_results.append(
            pool.name + ": " + str(round(pool.share, 2)) + "%"
        )

    return Page(state,[
        "Your Analysis Settings\n",
        "Top pools: " + str(state.top_count),
        "Threshold: " + str(state.threshold) + "%",
        "total Blocks Mined: " + str(total_blocks),
        "Mining Pool Shares:",
        *pool_results,
        Button ("Change Settings", "configure"),
    ])

@route
def return_home(state:State, top_count:int, threshold:float)->Page:
    return index(state)

@route
def reset_settings(state: State, top_count: int, threshold: float) -> Page:
    state.top_count = 3
    state.threshold = 50.0
    return configure(state)

@route
def results(state:State)-> Page:
    pass
```

```
@route
def pool_details(state:State)-> Page:
    pass

@route
def about(state:State)->Page:
    pass

start_server(State(3,50.0, [
    MiningPool("Pool A", 120, "Sample Week", 0.0),
    MiningPool("Foundry USA", 100, "Sample Week", 0.0),
    MiningPool("AntPool", 90, "Sample Week", 0.0),
    MiningPool("F2Pool", 75, "Sample Week", 0.0),
    MiningPool("ViaBTC", 65, "Sample Week", 0.0),
    MiningPool("SpierPool", 50, "Sample Week", 0.0),
```

```
]))
```

...



- `calculate_total_blocks()` adds up the blocks produced by every mining pool.
- `calculate_pool_shares()` calculates each pool's percentage of the total and prevents division by zero.

[redacted]

[redacted]

[redacted]

[redacted]

```
pool_results = []
```

```
for pool in state.pools:
```

```
  pools_results.append(
```

```
    pool.name + ": " + str(round(pool.share, 2)) + "%"
```

```
)
```

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

```
MiningPool("SpierPool", 50, "Sample Week", 0.0),
```

[redacted]

[redacted]

[redacted]

```
def calculate_pool_shares(pools: list[MiningPool]) -> None:
```

```
  total_blocks = calculate_total_blocks(pools)
```

```
  for pool in pools:
```

```
    if total_blocks > 0:
```

```
            pool.share = (pool.blocks_mined / total_blocks) * 100
```

```
    else:
```

```
        pool.share = 0.0
```

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

Total observed blocks

[redacted]



[redacted]

[redacted]



[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

```
def calculate_combined_share(pools: list[MiningPool],
```

```
                                top_count: int) -> float:
```

```
  combined = 0.0
```

```
# Your code goes here
```

```
  return combined
```



[redacted]

1. You'll need a loop to go through your mining pools.
2. You only want to add the first `top_count` pools, not every pool.
3. Each pool already has a `share` attribute.
4. Add those shares to `combined`.

[redacted]

[redacted]

[redacted]

[redacted]

this is my current code, i added some new functions and edited some. from drafter import *
from dataclasses import dataclass

```
set_website_title("Bitcoin Mining Decentralization Analyzer")
set_site_information(
    author="Jemuel Picol",
    description="Explore how bitcoin mining is distributed among mining pools and analyze mining concentration.",
    sources="Thanks to the Drafter docs and my study group.",
    planning=["CS1 Final Project 0 Pick a problem", "CS1 Final Project 1 Design"],
    links=[

"https://github.com/me50/jem-uel/blob/cs50/problems/2022/python/meal/CS1%20Final%20Project%200_%20Pick%20a%20Problem.pdf",

"https://github.com/me50/jem-uel/blob/cs50/problems/2022/python/meal/CS1%20Final%20Project%201_%20Design.docx"
        ]
)
hide_debug_information()
set_website_framed(False)

@dataclass
class MiningPool:
    name: str
    blocks_mined: int
    period: str
    share: float

@dataclass
class State:
    top_count: int
    threshold: float
    pools: list[MiningPool]
    selected_pool:str
    combined_share:float
```

```
def calculate_total_blocks(pools: list[MiningPool]) -> int:
    total = 0

    for pool in pools:
        total = total + pool.blocks_mined
    return total

def calculate_pool_shares(pools: list[MiningPool]) -> None:
    total_blocks = calculate_total_blocks(pools)

    for pool in pools:
        if total_blocks > 0:
            pool.share = (pool.blocks_mined / total_blocks) * 100
        else:
            pool.share = 0.0

def calculate_combined_share(pools: list[MiningPool],
                             top_count:int) -> float:
    combined = 0.0
    for pool in pools[:top_count]:
        combined = combined + pool.share
    return combined

def rank_pools(pools: list[MiningPool]) -> list[MiningPool]:
    ranked = sorted(pools, key=lambda pool: pool.blocks_mined, reverse=True)
    return ranked



def validate_inputs(top_count: int, threshold: float, pools: list[MiningPool]) -> str:
    if top_count < 1:
        return "Error: You must select at least one mining pool."

    if top_count > len(pools):
        return "Error: You selected more mining pools than are available."

    if threshold < 0 or threshold > 100:
        return "Error: Threshold must be between 0 and 100."

    return ""
```

```
@route
def index(state: State) -> Page:
    return Page(state, [
        "Bitcoin Mining Decentralization Analyzer\n",
        "Explore how bitcoin mining is distributed among mining pools and analyze mining
concentration.",
        Button("Start Analysis", "configure"),
    ])



@route
def configure(state:State) ->Page:
    return Page(state,[
        "Configure your Analysis",

        "Numer of largest mining pools:",
        TextBox("top_count", state.top_count),

        "Concentration threshold (%):",
        TextBox("threshold", state.threshold),

        Link("Return Home", "return_home"),
        Button("Run Analysis", "analyze"),
        Button("Reset Settings", "reset_settings"),
    ])

@route
def analyze(state:State, top_count:int, threshold:float)->Page:
    error = validate_inputs(top_count, threshold, state.pools)

    if error != "":
        return Page(state, [
            error,
            Button("Try Again", "configure"),
        ])

    state.top_count = top_count
    state.threshold = threshold
```

```
    total_blocks = calculate_total_blocks(state.pools)
    calculate_pool_shares(state.pools)

    state.pools = rank_pools(state.pools)

    state.combined_share = calculate_combined_share(
        state.pools, state.top_count
    )

    return results(state)

@route
def return_home(state:State, top_count:int, threshold:float)->Page:
    return index(state)

@route
def reset_settings(state: State, top_count: int, threshold: float) -> Page:
    state.top_count = 3
    state.threshold = 50.0
    return configure(state)

@route
def results(state:State)-> Page:
    pool_results = []

    for pool in state.pools:
        pool_results.append(
            pool.name + ": " + str(round(pool.share, 2)) + "%"
        )

    if state.combined_share > state.threshold:
        conclusion = "The combined share exceeds your threshold."
    elif state.combined_share == state.threshold:
        conclusion = "The combined share equals your threshold."
    else:
        conclusion = "The combined share is below your threshold."

    return Page(state, [
        "Bitcoin Mining Analysis Results",
        "Total Blocks: " + str(calculate_total_blocks(state.pools)),
```

```
        "Top Pools Selected: " + str(state.top_count),
        "Threshold: " + str(state.threshold) + "%",
        "Combined Share: " + str(round(state.combined_share, 2)) + "%",
        conclusion,
        "Mining Pool Rankings:",
        *pool_results,
        Button("Change Settings", "configure"),
        Button("About", "about"),
    ])


@route
def pool_details(state:State)-> Page:
    pass

@route
def about(state:State)->Page:
    pass

start_server(State(3,50.0, [
    MiningPool("Pool A", 120, "Sample Week", 0.0),
    MiningPool("Foundry USA", 100, "Sample Week", 0.0),
    MiningPool("AntPool", 90, "Sample Week", 0.0),
    MiningPool("F2Pool", 75, "Sample Week", 0.0),
    MiningPool("ViaBTC", 65, "Sample Week", 0.0),
    MiningPool("SpiderPool", 50, "Sample Week", 0.0),
    ],"Pool A", 0.0))
```

what are my next steps?

...



[redacted]

- Calculate total blocks mined.
- Calculate each mining pool's percentage.
- Rank mining pools from largest to smallest.
- Calculate the combined percentage of the top pools.
- Validate user inputs.
- Compare the combined percentage against the user's threshold.
- Display the results.

[redacted]

[redacted]



[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]



[redacted]



[redacted]

```
@route
```

```
def about(state: State) -> Page:
```

```
  pass
```

[redacted]

[redacted]

```
@route
```

```
def about(state: State) -> Page:i
```

```
  return Page(state, [
```

```
        "About This Analyzer",
```

```
# Explain the purpose of your website
```

```
# Explain how mining pool percentages are calculated
```

```
# Explain the limitations of your analysis
```

```
    Button("Back to Results", "results"),
```

```
])
```

[redacted]





[redacted]

[redacted]

[redacted]

```
selected_pool: str
```

[redacted]

[redacted]

```
SelectBox("field_name", options, default_value)
```

[redacted]



[redacted]

[redacted]

```
pool_names = []
```

```
for pool in state.pools:
```

```
  # Add the name of each pool to pool_names
```

[redacted]

[redacted]

```
"Select a mining pool:",
```

```
SelectBox("selected_pool", pool_names, state.selected_pool),
```

```
Button("View Pool Details", "choose_pool"),
```

[redacted]





[redacted]



[redacted]

[redacted]

```
for pool in state.pools:
```

```
  if pool.name == state.selected_pool:
```

```
    # You have found the selected pool
```









i used some of the vscode helper to get to where i am at rigth now, i created additional functions to go throught he pool's details and then i used vscode to help generate the table through drafter. this is the code i currently have as of rigth now i removed all the passes within the functions/routes. what are the next steps? from drafter import *
from dataclasses import dataclass

```
set_website_title("Bitcoin Mining Decentralization Analyzer")
set_site_information(
    author="Jemuel Picol",
    description="Explore how bitcoin mining is distributed among mining pools and
analyze mining concentration.",
    sources="Thanks to the Drafter docs and my study group.",
    planning=["CS1 Final Project 0 Pick a problem", "CS1 Final Project 1 Design"],
    links=[

"https://github.com/me50/jem-uel/blob/cs50/problems/2022/python/meal/CS1%20Final
%20Project%200_%20Pick%20a%20Problem.pdf",

"https://github.com/me50/jem-uel/blob/cs50/problems/2022/python/meal/CS1%20Final
%20Project%201_%20Design.docx"
        ]
)

hide_debug_information()
set_website_framed(False)

@dataclass
class MiningPool:
    name: str
    blocks_mined: int
    period: str
    share: float

@dataclass
class State:
    top_count: int
```

```
    threshold: float
    pools: list[MiningPool]
    selected_pool:str
    combined_share:float


def calculate_total_blocks(pools: list[MiningPool]) -> int:
    total = 0

    for pool in pools:
        total = total + pool.blocks_mined
    return total

def calculate_pool_shares(pools: list[MiningPool]) -> None:
    total_blocks = calculate_total_blocks(pools)

    for pool in pools:
        if total_blocks > 0:
            pool.share = (pool.blocks_mined / total_blocks) * 100
        else:
            pool.share = 0.0

def calculate_combined_share(pools: list[MiningPool],
                             top_count:int) -> float:
    combined = 0.0
    for pool in pools[:top_count]:
        combined = combined + pool.share
    return combined

def rank_pools(pools: list[MiningPool]) -> list[MiningPool]:
    ranked = sorted(pools, key=lambda pool: pool.blocks_mined, reverse=True)
    return ranked


def validate_inputs(top_count: int, threshold: float, pools: list[MiningPool]) -> str:
    if top_count < 1:
        return "Error: You must select at least one mining pool."

    if top_count > len(pools):
        return "Error: You selected more mining pools than are available."
```

```
    if threshold < 0 or threshold > 100:
        return "Error: Threshold must be between 0 and 100."

    return ""

@route
def index(state: State) -> Page:
    return Page(state, [
        "Bitcoin Mining Decentralization Analyzer\n",
        "Explore how bitcoin mining is distributed among mining pools and analyze mining
concentration.",
        Button("Start Analysis", "configure"),
        Link("About This Analyzer", "about"),
    ])



@route
def configure(state:State) ->Page:
    return Page(state,[
        "Configure your Analysis",
        "Numer of largest mining pools (1 to" + str(len(state.pools)) + "):",
        TextBox("top_count", state.top_count),
        "Concentration threshold (%):",
        TextBox("threshold", state.threshold),
        Button("Run Analysis", "analyze"),
        Button("Reset Settings", "reset_settings"),
        Link("Return Home", "return_home"),
    ])

@route
def analyze(state:State, top_count:int, threshold:float)->Page:
    error = validate_inputs(top_count, threshold, state.pools)
    if error != "":
        return Page(state, [
            "Invalid analysis settings: " + error,
            Button("Try Again", "configure"),
        ])
```

```
    state.top_count = top_count
    state.threshold = threshold
    calculate_pool_shares(state.pools)
    state.pools = rank_pools(state.pools)
    state.combined_share = calculate_combined_share(state.pools, state.top_count)
    return results(state)

@route
def results(state:State)-> Page:
    rows = []
    pool_names = []
    rank = 1
    for pool in state.pools:
        rows.append([rank, pool.name, pool.blocks_mined, str(round(pool.share, 2)) +
"%"])
        pool_names.append(pool.name)
        rank = rank + 1

    if state.combined_share > state.threshold:
        conclusion = "The selected pools' combined share exceeds your threshold."
    elif state.combined_share == state.threshold:
        conclusion = "The combined share is exactly equal to your threshold."
    else:
        conclusion = "The selected pools' combined share is below your threshold."

    return Page(state, [
        "Bitcoin Mining Analysis Results",
        "Total observed blocks: " + str(calculate_total_blocks(state.pools)),
        "Top pools selected: " + str(state.top_count),
        "Threshold: " + str(state.threshold) + "%",
        "Combined share: " + str(round(state.combined_share, 2)) + "%",
        conclusion,
        "Mining Pool Rankings",
        Table(rows, ["Rank", "Pool", "Blocks Mined", "Share"]),
        "Select a mining pool to inspect:",
        SelectBox("selected_pool", pool_names, state.selected_pool),
        Button("View Pool Details", "choose_pool"),
        Button("Change Settings", "configure"),
        Link("About This Analyzer", "about"),
    ])
```

```
@route
def choose_pool(state: State, selected_pool: str) -> Page:
    state.selected_pool = selected_pool
    return pool_details(state)

@route
def pool_details(state: State) -> Page:
    for pool in state.pools:
        if pool.name == state.selected_pool:
            return Page(state, [
                "Mining Pool Details",
                "Pool name: " + pool.name,
                "Blocks mined: " + str(pool.blocks_mined),
                "Observation period: " + pool.period,
                "Share of observed blocks: " + str(round(pool.share, 2)) + "%",
                Button("Back to Results", "results"),
            ])

    return Page(state, [
        "The selected mining pool was not found.",
        Button("Back to Results", "results"),
    ])

@route
def about(state: State, selected_pool: str = "") -> Page:
    return Page(state, [
        "About This Analyzer",
        "Purpose: This website helps visitors explore how observed Bitcoin blocks are
distributed among mining pools.",
        "How pool shares are calculated: Divide a pool's block count by the total blocks in
the sample, then multiply by 100.",
        "How combined share is calculated: Rank pools by blocks mined and add the
shares of the largest number of pools chosen by the user.",
        "Threshold comparison: The site compares this combined percentage with the
visitor's selected threshold.",
        "Limitations: The current block counts are fictional sample data, not live Bitcoin
statistics. Mining pool concentration alone cannot prove coordinated control of the
Bitcoin network.",
        Button("Back to Results", "results"),
```

```
        Link("Return Home", "index"),
    ])

@route
def return_home(state:State, top_count:int, threshold:float)->Page:
    return index(state)

@route
def reset_settings(state: State, top_count: int, threshold: float) -> Page:
    state.top_count = 3
    state.threshold = 50.0
    return configure(state)

start_server(State(3,50.0, [
    MiningPool("Pool A", 120, "Sample Week", 0.0),
    MiningPool("Foundry USA", 100, "Sample Week", 0.0),
    MiningPool("AntPool", 90, "Sample Week", 0.0),
    MiningPool("F2Pool", 75, "Sample Week", 0.0),
    MiningPool("ViaBTC", 65, "Sample Week", 0.0),
    MiningPool("SpiderPool", 50, "Sample Week", 0.0),
```

```
],"Pool A", 0.0))
```

...

[redacted]





[redacted]

[redacted]





[redacted]

[redacted]

[redacted]

```
SelectBox("selected_pool", pool_names, state.selected_pool),
```

```
Button("View Pool Details", "choose_pool"),
```

```
Button("Change Settings", "configure"),
```

```
Link("About This Analyzer", "about"),
```

[redacted]

[redacted]





+1

[redacted]

[redacted]

[redacted]



[redacted]

[redacted]

[redacted]

[redacted]

[redacted]



[redacted]

```
assert_equal(
```

```
  calculate_total_blocks([
```

```
    MiningPool("Test A", 10, "Test", 0.0),
```

```
    MiningPool("Test B", 20, "Test", 0.0)
```

```
  ]),
```

)

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]



[redacted]



[redacted]

```
def calculate_total_blocks(pools: list[MiningPool]) -> int:
```

```
"""
```

```
  Calculates the total number of blocks in the mining pool dataset.
```

```
"""
```

[redacted]



[redacted]

[redacted]

[redacted]



[redacted]



[redacted]

[redacted]

[redacted]



[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]



